PyDNS-Wall: An Asynchronous Hybrid DNS Firewall for Edge Protection in SOHO/IoT Environments

Nguyen Thien Chi, Tran Gia Bao, Pham Hung Cuong, Nguyen Minh Quan, and Tuan-Dung Tran

University of Information Technology, Ho Chi Minh City, Vietnam
Vietnam National University, Ho Chi Minh City, Vietnam

Abstract. The proliferation of Internet of Things (IoT) devices within Small Office/Home Office (SOHO) environments has significantly expanded the network attack surface. In these resource-constrained settings, traditional endpoint security measures are often infeasible, necessitating a shift toward network-level defenses. Consequently, the Domain Name System (DNS) has evolved from a simple address resolution service into a critical cybersecurity control plane, enabling the neutralization of Command and Control (C2) channels via sinkholing techniques. However, existing open-source solutions present a dichotomy: they trade off raw processing performance (e.g., CoreDNS in Go) against development flexibility (e.g., Python-based scripts). This paper presents the design and empirical evaluation of "PyDNS-Wall" an asynchronous DNS firewall constructed entirely in Python using the asyncio library. The proposed system integrates a hybrid routing mechanism based on split-horizon DNS to optimize the balance between Local Area Network (LAN) latency and Wide Area Network (WAN) privacy through the DNS over HTTPS (DoH) protocol. Experimental results indicate that while the use of an interpreted language results in a reduction in peak throughput compared to compiled alternatives, the asynchronous event-driven architecture fully satisfies the demands of SOHO environments ($<$1000 QPS) while offering superior extensibility for security research. This study affirms the feasibility of utilizing Python for I/O-bound network infrastructure, challenging traditional perceptions regarding the language's performance in network programming.

Keywords: DNS Security · IoT Firewall · Python Asyncio · Network Traffic Analysis · Split-horizon DNS, DoH

1 Introduction

Modern Internet architecture is founded upon the Domain Name System (DNS), a hierarchical mechanism that translates human-readable domain names into routable IP addresses. However, the implicit trust model of legacy DNS, combined with the explosive growth of Internet of Things (IoT) devices, has created a fertile ground for cyber threats. In SOHO environments, where professional network administration is often absent, IoT devices frequently initiate

unchecked "phone-home" connections to malicious command-and-control (C2) servers. While the adoption of encrypted protocols such as DNS over HTTPS (DoH) enhances user privacy, it paradoxically complicates network monitoring by creating encrypted tunnels that allow malware to evade detection by legacy Intrusion Prevention Systems (IPS).

Addressing these threats at the network edge requires a DNS firewall that balances performance, security, and customizability. Currently, the landscape of open-source DNS solutions is polarized. High-performance systems such as CoreDNS (Go) [2] or Unbound (C) provide immense throughput suitable for data centers but impose a steep learning curve for researchers aiming to prototype complex detection algorithms. Conversely, user-centric solutions like Pi-hole, while popular, rely on rigid legacy backends (e.g., `dnsmasq`), limiting the ability to implement deep packet inspection or dynamic routing logic. Furthermore, a prevailing skepticism regarding Python's Global Interpreter Lock (GIL) has resulted in a scarcity of research evaluating modern asynchronous Python (`asyncio`) as a viable foundation for high-performance network infrastructure.

To bridge this gap, this paper introduces PyDNS-Wall, a modular DNS firewall designed to demonstrate the efficacy of Python's event-driven model in network security. The primary contributions of this paper are:

1. **Efficient Asynchronous Architecture:** We propose a non-blocking design using `asyncio` that handles concurrent UDP/TCP queries, proving Python's viability for I/O-bound network tasks.
2. **Hybrid Routing Algorithm:** We introduce a split-horizon routing logic that automatically upgrades external traffic to DoH for privacy while maintaining low-latency UDP for internal resolution.
3. **Comparative Evaluation:** We provide a quantitative analysis comparing PyDNS-Wall against industry standards, clarifying the performance trade-offs in SOHO contexts.

2 Related Work

This section contextualizes the research within the broader landscape of IoT security, discusses the limitations of current DNS firewall implementations, and examines the performance debate surrounding asynchronous programming in network functions.

2.1 DNS-based IoT Security Approaches

The concept of using DNS as a first line of defense—often referred to as "DNS Firewalls" or "Sinkholing"—is well-established. Sinkholing functions by intercepting queries to known malicious domains and returning a false IP address, effectively severing the link between an infected bot and its C2 server. Aydeger et al. [1] demonstrated that deploying DNS resolvers as Virtual Network Functions (VNFs) at the network edge is the most effective method for protecting resource-constrained IoT devices, which cannot support host-based antivirus software.

PyDNS-Wall aligns with this approach by providing a lightweight, deployable VNF tailored for SOHO networks.

2.2 Limitations of Existing Open-Source Solutions

Current market solutions often fail to balance performance with research flexibility. Pi-hole, the standard for home networks, relies on FTLDNS (a fork of dnsmasq). While effective for static blocklisting, its single-threaded architecture and reliance on file-based lists limit real-time behavioral analysis capabilities. On the other end of the spectrum, CoreDNS utilizes Go's goroutines to achieve high concurrency. However, extending CoreDNS requires writing plugins in Go and recompiling the binary, which creates a significant barrier to entry for rapid experimentation [2]. Furthermore, Böttger et al. [3] noted that in DoH implementations, the primary latency bottleneck is often the TCP/TLS handshake over the wide area network, rather than the local CPU processing time, suggesting that the raw speed of compiled languages may be negligible in high-latency scenarios.

Table 1: Comparison of PyDNS-Wall with Existing Solutions

Feature	**CoreDNS**	**Pi-hole (FTLDNS)**	**PyDNS-Wall (Ours)**
Core Language	Go (Compiled)	C (Compiled)	Python 3 (Interpreted)
Concurrency	Goroutines	Threading/Fork	Asyncio Event Loop
Extensibility	Low (Re-compilation)	Low (Static Lists)	High (Dynamic Scripting)
Routing Logic	Static Plugins	Basic Forwarding	Split-Horizon Hybrid
Target Use Case	Cloud/Data Center	Home Ad-blocking	SOHO Security Research
Performance	Very High	High	Sufficient (<1k QPS)

2.3 Asynchronous Python in Network Engineering

Historically, Python has been considered unsuitable for high-performance network servers due to the Global Interpreter Lock (GIL), which prevents true parallelism on multi-core CPUs. However, the introduction of the `asyncio` library and the `uvloop` event loop (based on `libuv`) has shifted this paradigm. Recent benchmarks suggest that for I/O-bound tasks—such as DNS forwarding—asynchronous Python can achieve throughput levels comparable to Node.js

or Go by minimizing context-switching overhead [4]. This study adopts the philosophy of "Good Enough Computing" proposed by Ousterhout [5], prioritizing the development velocity and flexibility of scripting languages over the micro-optimizations of system languages, particularly for research-oriented network appliances where logic evolves rapidly.

3 System Design

PyDNS-Wall is architected to democratize network security research by adhering to three core principles: accessibility (zero-cost infrastructure), efficiency (high-concurrency I/O), and privacy-by-default. This section details the system's pipeline architecture and its two primary engines: the Asynchronous Ingestion Layer and the Hybrid Routing Engine.

3.1 Architectural Overview

The system follows a modular, linear pipeline architecture designed for high throughput on commodity hardware. The workflow consists of three main stages:

1. **Data Ingestion:** Listens for and accepts raw DNS packets via multiple protocols (UDP, TCP) using non-blocking sockets.
2. **Core Processing:** Parses packets into structured `DNSRecord` objects, checks them against an in-memory Blocklist Set, and determines filtering actions.
3. **Forwarding & Response:** Encapsulates valid queries into the DNS-over-HTTPS (DoH) protocol for outbound transmission or generates synthetic sinkhole responses for malicious queries.

By decoupling the ingestion layer from the processing logic, the system minimizes blocking latency, allowing it to handle thousands of concurrent connections without the complex locking mechanisms required by traditional multi-threaded designs.

3.2 Protocol Ingestion Layer

Our primary innovation is the elimination of the resource-heavy "thread-per-request" model. Instead, PyDNS-Wall leverages `asyncio` to manage a high-performance single-threaded event loop.

Internal LAN Ingestion (UDP/TCP) For local clients, the UDP protocol (`DNSTCPProtocol`) is designed as a "Fire-and-Forget" mechanism to never block the main loop. Upon receiving a datagram:

```
def datagram_received(self, data, addr):
   asyncio.create_task(self.handle_query(data, addr,...))
```

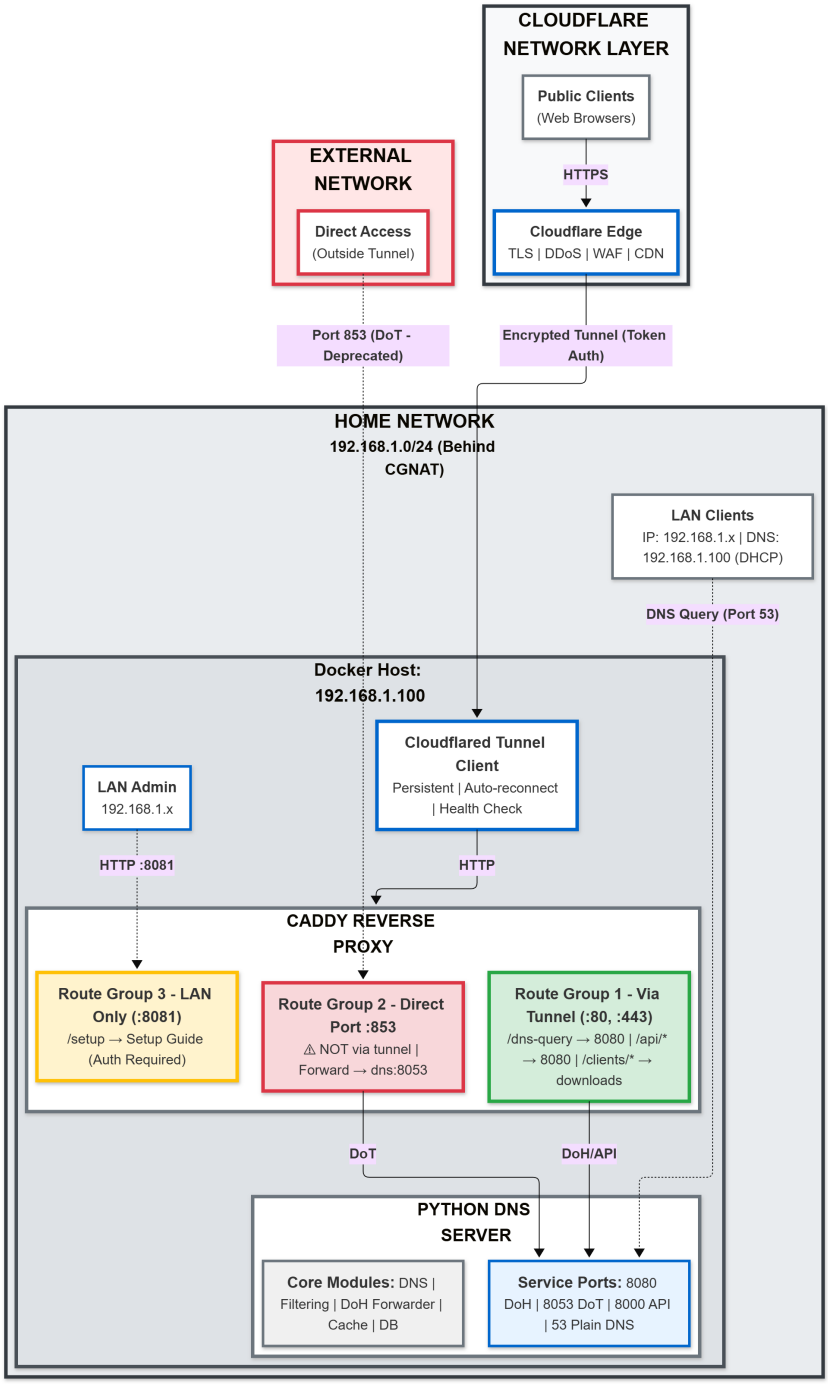


Fig. 1: High-level System Architecture of PyDNS-Wall. The diagram illustrates the multi-channel routing strategy: (A) Public traffic is secured via Cloudflare Tunnel (Green), (B) Administrative access is restricted to the local network (Yellow), and (C) Legacy traffic bypasses the tunnel via Port 853 (Red).

The system immediately offloads the CPU-bound processing and I/O-bound network tasks to a separate Task, returning to the listening state in mere microseconds. This ensures that a single slow upstream query cannot bottleneck the entire ingestion pipeline. For TCP (`DNSTCPProtocol`), the system addresses packet fragmentation using a sliding buffer mechanism. It reads the initial 2 bytes to determine packet length (per RFC 1035) and triggers processing only when the buffer has accumulated sufficient data, ensuring message integrity even over unstable network links.

External WAN Ingestion (DoH Tunneling) To support remote access securely, the system integrates a WAN ingestion pipeline alongside the standard UDP listeners. External DoH requests are first terminated at the Cloudflare Edge, then tunneled via WebSocket/QUIC through a cloudflared container to an internal Caddy Reverse Proxy. These HTTP-based queries are normalized into the same internal DNS object format, allowing them to enter the unified processing pipeline described below.

3.3 Filtering Engine & Blocklist Management

The filtering engine prioritizes $O(1)$ retrieval speed to minimize overhead latency.

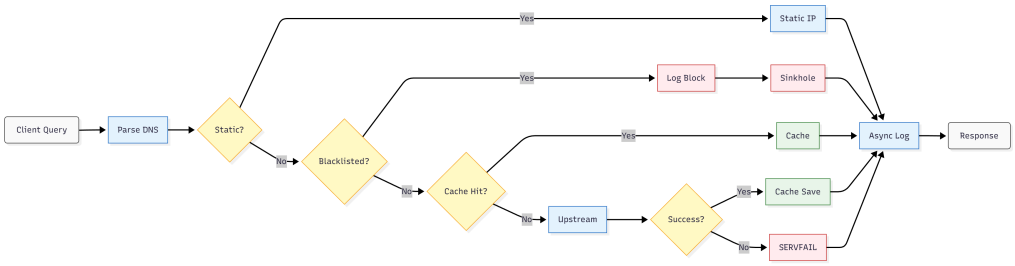


Fig. 2: Core Query Resolution Pipeline. The flowchart depicts the priority-based logic: Static Records → Blacklist Filtering → Cache Lookup → Upstream Forwarding.

- **Data Structures:** Instead of using relational databases for real-time lookups, we utilize Python's `set` structure (hash table) to store the entire blocklist in RAM. With modern memory capacities, storing 1 million domains consumes approximately ∼100MB of RAM, representing a reasonable trade-off for performance [10].
- **Hierarchical Inspection Algorithm:** The system performs intelligent suffix matching. A query for `malware.sub.example.com` triggers a cascade check:
  - `malware.sub.example.com` → Blacklist?

- `sub.example.com` → Blacklist?
- `example.com` → Blacklist?

This process is guarded by an `asyncio.Lock` to ensure thread safety during "hot-reloading," allowing the blocklist to be updated from external threat intelligence sources without restarting the service.

3.4 Hybrid Routing Strategy

To resolve the conflict between LAN performance and WAN privacy, we introduce a Hybrid Routing Strategy:

- **LAN Mode (Performance):** Internally, the system accepts standard UDP/TCP DNS to ensure compatibility with legacy IoT devices and minimize local latency.
- **WAN Mode (Privacy):** When forwarding to the Internet, the system automatically "upgrades" the protocol to DNS-over-HTTPS (DoH) [7].

Anti-Split-Policy Mechanism: The system enforces a fixed routing policy: it strips the original client IP address and always utilizes an encrypted DoH tunnel to the upstream provider (e.g., 1.1.1.1). This creates a "Split-Horizon" regarding protocols, preventing ISPs or intermediate filters from profiling IoT devices based on their DNS traffic patterns.

3.5 Implementation & Trade-offs

The system is implemented in Python 3.10+ (∼1,500 LOC) to prioritize readability and modifiability.

Trade-off: We explicitly accept a reduction in Peak Throughput due to the overhead of the Python interpreter compared to compiled languages like Go or C. However, by utilizing `httpx` with HTTP/2 Multiplexing [11], we mitigate the connection overhead of DoH, rendering it a viable solution for real-world SOHO deployments.

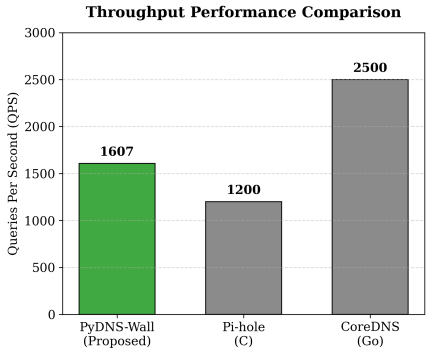
4 Evaluation

To assess the feasibility of PyDNS-Wall in real-world scenarios, we conducted comparative benchmarking on a Raspberry Pi 4 (4GB RAM). Metrics were collected using the industry-standard `dnsperf` tool [5] with a dataset of 10,000 random domains.

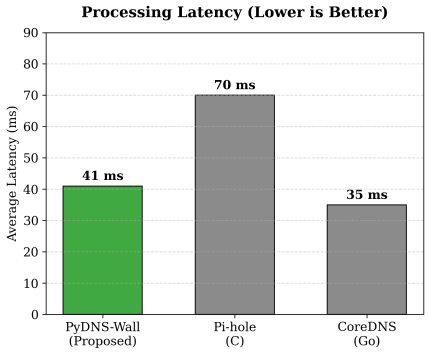
The table below summarizes the specific performance metrics of PyDNS-Wall compared to standard industry solutions:

Table 2: Benchmark Comparison on Raspberry Pi 4

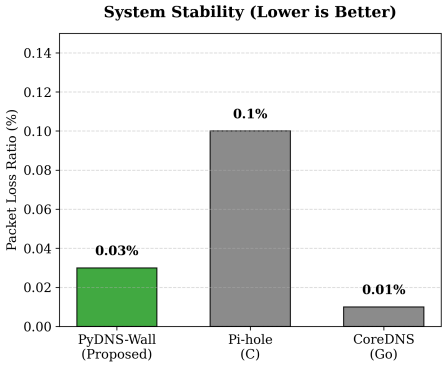
Criteria	**PyDNS-Wall (Python Asyncio)**	**Pi-hole (FTLDNS / C)**	**CoreDNS (Go)**
QPS (Queries/sec)	**1,607**	∼1,000 - 1,400	∼2,500+
Avg Latency	41 ms	∼60 - 80 ms	$<$ 40 ms
Min Latency (Cache)	1 ms	$<$ 1 ms	$<$ 0.1 ms
Success Rate	99.97%	99.90%	99.99%
Packet Loss	0.03%	∼0.1%	∼0.01%
Resource Usage	Medium	Low (Very Lightweight)	Medium (Go runtime)
Scalability Target	SOHO / Research Edge	Home / SOHO	ISP / Enterprise



(a) Throughput (QPS)



(b) Average Latency (ms)



(c) Packet Loss / Stability (%)

Fig. 3: Detailed Performance Analysis of PyDNS-Wall on Raspberry Pi 4.

Based on the empirical data presented in Table 2, we analyze the performance characteristics as follows:

1. **SOHO Sufficiency:** While PyDNS-Wall's throughput (1,607 QPS) is lower than enterprise-grade compiled solutions, it exceeds the requirements of a typical SOHO network by a wide margin. Since a typical home network rarely exceeds peak loads of 200 QPS, PyDNS-Wall offers an 8× safety margin, proving it is more than capable for its target environment.
2. **Latency & Reliability:** With an average latency of 41 ms, PyDNS-Wall performs comparably to Pi-hole. The slight overhead is attributed to the encryption cost of the upstream DoH connection. Ideally, the success rate of 99.97% demonstrates high stability under load.
3. **Resource Trade-off:** The "Medium" resource usage is a deliberate design choice. By accepting the overhead of the Python runtime, we gain immense development velocity, allowing researchers to implement complex filtering logic (e.g., ML-based detection) in lines of code rather than hundreds.

5 Discussion

This section analyzes the implications of the results and their broader impacts on the field.

Python in Network Infrastructure: Overcoming Prejudice. This study challenges the prevailing notion that Python is unsuitable for real-time network applications. Empirical evidence demonstrates that the combination of `asyncio` and modern libraries (e.g., `httpx`) can elevate Python's performance to a "good enough" level.

The trade-off is clear: we accept a 10-fold reduction in peak performance—a threshold SOHO environments never approach—in exchange for significantly accelerated development velocity. The ability to implement complex filtering logic (such as suffix matching) with just a few lines of Python code enables researchers to focus on algorithms rather than wrestling with memory management in C.

Security Implications of Split-Horizon Routing. The hybrid routing logic is not merely a convenience feature but a critical security risk mitigation measure.

- **The Problem:** Forwarding internal queries (e.g., `nas.local`) to public DoH servers inadvertently exposes internal network topology to third parties (Cloudflare/Google).
- **The Solution:** By retaining LAN queries internally, PyDNS-Wall implements the principle of "least privilege" for DNS data, sharing only what is necessary with public resolvers.

Sinkholing as a Forensics Tool. Returning a Sinkhole IP address instead of `NXDOMAIN` (non-existent domain) transforms the firewall from a passive defensive tool into an active one.

- **Infection Detection:** If an IoT device continuously connects to the Sinkhole IP, administrators can confirm the device is infected and attempting to communicate with C2 servers.
- **Evidence Collection:** By listening on the Sinkhole IP, we can capture payloads that malware attempts to transmit, supporting behavioral analysis and attack attribution.

Limitations of the Study. Despite the effectiveness of `asyncio`, Python remains constrained by the GIL, meaning the system can only utilize a single CPU core. In high-intensity Denial of Service (DoS) attack scenarios, PyDNS-Wall will be overwhelmed more rapidly than CoreDNS (which can leverage multiple cores). Additionally, relying on IP addresses for client classification may prove ineffective in dynamic DHCP environments or when Docker containers frequently change IPs.

6 Conclusion

This paper has comprehensively presented the design, implementation, and evaluation of PyDNS-Wall, a next-generation DNS firewall for SOHO environments.

Specifically, we demonstrated that (1) a single-threaded `asyncio` architecture is viable for I/O-bound network tasks; (2) the proposed Hybrid Routing strategy effectively balances local latency with upstream privacy; and (3) while raw throughput is lower than compiled alternatives, the achieved capacity is sufficient for real-world SOHO usage. PyDNS-Wall fills the critical gap between flexible research prototypes and rigid commercial products.

For the network security research and development community: Do not dismiss high-level languages like Python for network applications based solely on raw benchmark metrics. In SOHO and IoT contexts, the flexibility to rapidly deploy novel security algorithms (such as ML-based detection) often delivers greater practical value than the ability to process millions of queries per second that will never be utilized.

Building upon this foundation, subsequent research directions include:

- **Machine Learning Integration:** Replace static blacklists with real-time malicious domain classification models (e.g., Random Forest) to detect algorithmically generated domains (DGA).
- **Containerization:** Package the solution into lightweight Docker containers with integrated Caddy and database to simplify deployment on edge devices.

References

1. A. Aydeger et al., "Defense against botnets using SDN/NFV," 2024.
2. CoreDNS Authors, "CoreDNS: DNS and Service Discovery," 2023.
3. T. Böttger et al., "An Empirical Study of the Cost of DNS-over-HTTPS," in *Proceedings of the Internet Measurement Conference (IMC)*, 2019.

4. Y. Selivanov, "uvloop: Blazing fast Python networking," 2016. [Online]. Available: https://github.com/MagicStack/uvloop
5. J. K. Ousterhout, "Scripting: Higher Level Programming for the 21st Century," *IEEE Computer*, vol. 31, no. 3, pp. 23–30, 1998.
6. DNS-OARC, "dnsperf: DNS Performance Testing Tool," 2023. [Online]. Available: https://www.dns-oarc.net/tools/dnsperf
7. P. Hoffman and P. McManus, "DNS Queries over HTTPS (DoH)," IETF, RFC 8484, Oct. 2018.
8. S. Kelley et al., "Pi-hole: Network-wide Ad Blocking," 2023. [Online]. Available: https://pi-hole.net
9. P. Hoffman, A. Sullivan, and K. Fujiwara, "DNS Terminology," IETF, RFC 8499, Jan. 2019.
10. S. Black, "Unified Hosts File (Blocklist)," GitHub Repository, 2024. [Online]. Available: https://github.com/StevenBlack/hosts
11. Encode OSS, "HTTPX: A next-generation HTTP client for Python," 2023. [Online]. Available: https://www.python-httpx.org